

Surplus.

A local food exchange that reduces waste through trusted, invite-only sharing of surplus produce.

Kennan Kaneshiro · Shea Stevens · Matt Ong · Kim Cates · Jeff Wu

PHASE	TEAM	TERM
Project Initiation	Team 4	SU2026

Problem & Motivation

Edible surplus food is discarded every day, yet there is no lightweight, trusted way for a small community to redistribute it before it spoils.

- Households, gardeners, and small growers regularly have **more than they can use**, with no easy outlet.
- Open public marketplaces raise **trust, safety, and accountability** concerns around food handed between strangers.
- Many groups in local neighborhoods and communities **lack a tool** built for invite-only sharing and coordination.

Project Overview

A responsive web app where invited members browse listings and request surplus food within a private community.

- **Invite-only access:** members join by invitation and create a profile.
- **Listings** show category, quantity, dietary / allergen tags, and a pickup window.
- **Browse, search, and filter** active listings, then request a quantity.
- **Private message threads** and notifications coordinate each exchange.
- **A request queue** handles requests in order and prevents conflicts.

Project Objectives

WHAT WE WILL ACHIEVE

- Deliver a **responsive web app** for invite-only surplus food exchange.
- Implement the **request queue and lifecycle** with quantity-integrity and state rules.
- Enforce **role-based access** and permission rules.
- Ship **automated tests** , a clear README, and seeded demo data.
- Practice the **full SDLC** as a collaborating team.

VALUE DELIVERED

- **Community & end users:** redistribute edible surplus before it spoils.
- **Trust & safety:** invite-only access and a fair request queue keep exchanges accountable.
- **Convenience:** one place to find, request, and coordinate exchanges.
- **The team:** hands-on practice across the full development lifecycle.

Project Scope

IN SCOPE

- Invite-only registration and login, plus member profiles.
- Browse, search, and filter active listings.
- Request queue to ensure requests are handled in order without conflicts.
- Database schema as a single source of truth for listings, threads, users, reviews, and more.
- Private message thread for each exchange.

OUT OF SCOPE

- Payments, prices, tips, or any paid transactions. This exchange is free.
- Public sign-up. Access stays invite-only.
- “Looking for” posts: posters only offer up what they have.
- Production-grade scaling, load handling, and security hardening.
- Customer support.

User Roles / Personas

UNREGISTERED

Guest

Holds an invite token but has not registered yet. A Guest can only register.

REGISTERED USER

Member

A registered, active user. A Member can browse, search, and filter listings, manage a profile, submit requests, and send messages.

LISTING OWNER

Poster

The Member who owns a listing. A Poster handles that listing's request queue and messages the recipient. In this scope slice, active listings come from seeded data; creating listings is out of the current scope (see Section 6).

REQUESTER

Recipient

The Member who submits a request for an item and, if approved, receives it. The poster reaches the recipient through the message thread.

ADMINISTRATOR

Admin

A privileged operator who keeps the community safe and trusted. An Admin manages invitations, moderates listings and content, suspends users, and deactivates listings without deleting audit history.

Key Use Cases & User Stories

US-01 · UC-01 · Register

As a Guest, I want to **register with an invite token**, so I can join the trusted community.

US-06 · UC-06 · Browse & filter

As a Member, I want to **browse, search, and filter** active listings, so I can find items of interest.

US-08 · UC-08 · Submit request

As a Recipient, I want to **submit a request** for a quantity, so it joins the listing's queue.

US-22 · UC-21 · Suspend a user

As an Admin, I want to **suspend a member** account, so a misbehaving user can no longer log in or act.

US-09 · UC-09 · View queue

As a Poster, I want to see my listing's **pending requests in order**, so I can handle them fairly.

US-10 · UC-10 · Approve / deny

As a Poster, I want to **approve or deny** the next request, so I control distribution without conflicts.

US-12 · UC-12 · Message thread

As a Member, I want to **message the other party** in the exchange thread, so we can coordinate.

US-23 · UC-22 · Deactivate a listing

As an Admin, I want to **deactivate any listing**, so I can hide it from browsing without deleting audit history.

Acceptance Criteria

User story · US-10

Approve or deny the next request

Source UC-10

Actor Poster

Priority High

“As a Poster, I want to approve or deny a pending request on my listing, so I control who receives my items, in order, without conflicts.”

APPROVE

Given a **REQUESTED** request and approved qty ≤ remaining, when the Poster approves, then it becomes **APPROVED** , the remaining quantity decreases, and the Recipient is notified.

DENY

Given a **REQUESTED** request, when the Poster denies it, then it becomes **DENIED** and the Recipient is notified.

CONFLICT PREVENTED

Given an approval would push the approved quantity past the remaining quantity, when the Poster tries to approve, then the system **rejects it** and nothing changes.

PERMISSION

Given a Member who does **not** own the listing, when they try to approve or deny, then access is **denied** and nothing changes.

Request Lifecycle

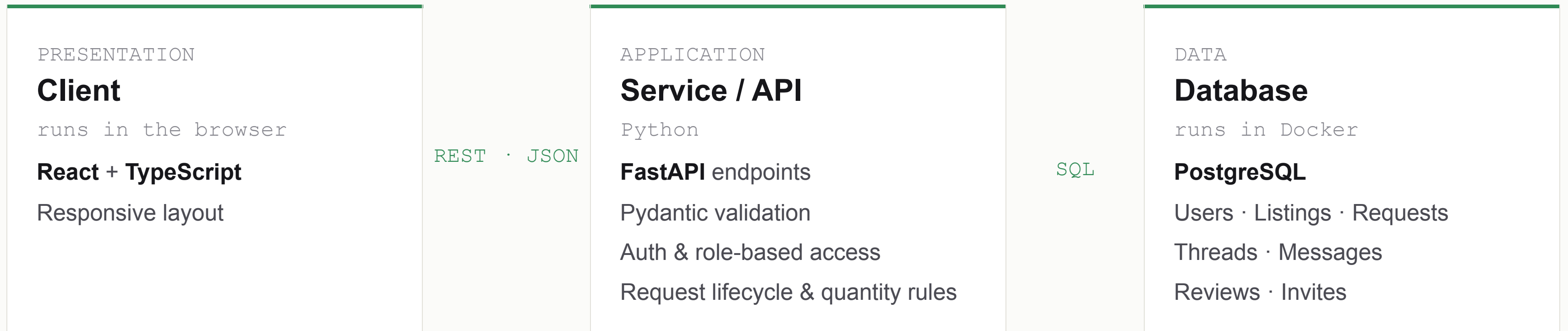


REQUESTED → **DENIED** Terminal rejection:
a request can be
declined before it is
approved.

→ **CANCELLED** Allowed only from REQUESTED
or APPROVED; cancelling after
pickup is blocked.

Quantity rule Approved quantity can
never exceed the
listing's remaining
quantity.

System Architecture



Technology Stack

FRONTEND

React	UI
TypeScript	typing
Vite	build / dev
Node.js + npm	deps

BACKEND & DATA

FastAPI	web framework
PostgreSQL	database
pytest	unit tests

TOOLING & DEVOPS

GitHub	VCS · PRs · CI
GitHub Actions	automated linting
Docker	packaging
ESLint · Ruff	linting

Development Methodology

We work in **one-week iterations** aligned to the course schedule. Each sprint runs a full slice of the lifecycle and ends with a working, demonstrable increment.



↺ repeat each sprint · incremental, iterative delivery

Project Management Overview

ROLES & RESPONSIBILITIES

Kennan Kaneshiro Contributor	Connect the frontend to the API and database (Python, SQL, TypeScript); project documentation.
Shea Stevens DBA	Schema design, ownership, and administration; front-end design support.
Matt Ong QA Lead	Test planning and bug tracking; design and test-tooling review.
Jeff Wu Contributor	Dev environment setup, npm build scripts, and GitHub Actions CI; full-stack feature work.
Kim Cates Contributor	Feature development across the stack; responsibilities to be confirmed.

MEETINGS & MINUTES

Recurring **weekly** syncs, evenings or weekends per availability

Decisions reached by **consensus**; area lead breaks ties

Minutes kept in a shared **Google Docs** folder

Each entry logs **decisions, action items, and owners**

TASK MANAGEMENT & TRACKING

Every story, task, and bug is a **GitHub Issue** with an owner

Projects board: To Do → In Progress → In Review → Done

Issues grouped by **milestone** ; work on dedicated branches

Pull requests reviewed before merging to main

Quality Assurance Plan

TESTING APPROACH

Unit tests with each feature (Vitest, pytest)

Permissions & role tests

End-to-end manual/automated tests

DEFECT TRACKING

Bugs filed as **GitHub Issues**

Severity & priority labels

Triaged in the weekly sync

Fixed via branch → PR → verify

CODE QUALITY

Mandatory PR review (≥ 1 approver)

GitHub Actions lint gates (ESLint, Ruff)

Key Risks & Mitigation Strategies

RISK	IMPACT	MITIGATION STRATEGY
Scope creep	High	Lock an MVP; defer extras to the backlog; review scope each sprint.
Uneven workload / availability	Medium	Clear roles, small tasks, and pairing on critical paths.
Frontend / backend integration drift	Medium	Agree the API schema early; mock endpoints.
Concurrent-request quantity bug	High	Database constraints + transactions + targeted tests.
Local env setup issues	Medium	Get everything working locally in Sprint 1

Project Artifacts

REQUIREMENTS

Requirements Spec

User stories, acceptance criteria, and scope definition.

DESIGN

Technical Design Doc

Domain model diagram, database schema, solution design document

TESTING

Test Plan & Cases

Unit, integration, and end-to-end cases, including edge cases.

DEPLOYMENT

Deployment Guide

Cross-platform Environments, GitHub Actions.

AUTOMATION

npm Scripts

One-command local tasks: setup, dev, test, lint, and more.

DOCS

README & Setup Guide

Install steps, seeded demo data, and how to run the app.

PROCESS

Board & Minutes

GitHub Projects board and the meeting-minutes log.

Project Timeline



Milestones

M1 Week 3 · Jun 9	Inception Inception presentation, core requirements, and the requirements packet (due Jun 8).
M2 Week 4 · Jun 16	Requirements review Requirements review presentation and the peer-review report (due Jun 15).
M3 Week 6 · Jul 2	Technical design Technical design document: UML, ER / database schema, and API contracts (due Jul 1).
M4 Week 7 · Jul 7	Release 1 demo R1 presentation and the first working release; manual test cases follow in Week 8.
M5 Week 10 · Jul 28	Release 2 demo R2 presentation and the second working release, with R1 retrospective fixes.
M6 Week 11 · Aug 6	Deployment Deployment guide and a deployed application.
M7 Week 12 · Aug 11	Final delivery Final presentation, final submission, peer evaluation, and individual reflection.

Next Steps

Incorporate **inception feedback** into the requirements packet.

Complete the **Technical Design Document** (Week 6, Jul 1).

Deliver the first working release at the **R1 demo** (Week 7, Jul 7).

Prepare the **Requirements Review** presentation (Week 4, Jun 16).

Begin the **R1 build** with one-week iterations.

Thank you.

Questions & discussion
welcome.